

1 Singly Linked List (SLL)

Structure

C

 Copy code

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

◆ Insert at Beginning

C

 Copy code

```
struct Node* insertBegSLL(struct Node*  
head, int data) {  
    struct Node* newNode = (struct  
Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = head;    // new node  
points to old head  
    return newNode;        // new node  
becomes head  
}
```



◆ Insert at Specific Position (1-based)

C

 Copy code

```
struct Node* insertPosSLL(struct Node*
head, int data, int pos) {
    if (pos == 1)
        return insertBegSLL(head, data);

    struct Node* temp = head;
    struct Node* newNode = (struct
Node*)malloc(sizeof(struct Node));
    newNode->data = data;

    for (int i = 1; i < pos - 1 &&
temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL)
        return head;    // invalid
position

    newNode->next = temp->next;
    temp->next = newNode;
    return head;
}
```

◆ Insert at End

C

 Copy code

```
struct Node* insertEndSLL(struct Node*  
head, int data) {  
    struct Node* newNode = (struct  
Node*)malloc(sizeof(struct Node));  
    newNode->data = data;  
    newNode->next = NULL;  
  
    if (head == NULL)  
        return newNode;  
  
    struct Node* temp = head;  
    while (temp->next != NULL)  
        temp = temp->next;  
  
    temp->next = newNode;  
    return head;  
}
```

C

 Copy code

```
void insertAfterSLL(struct Node* head,
int key, int data) {
    struct Node* temp = head;

    // Traverse list to find the given
node (key)
    while (temp != NULL && temp->data !=
key)
        temp = temp->next;

    // If given node not found
    if (temp == NULL) {
        printf("Node not found\n");
        return;
    }

    // Create new node
    struct Node* newNode = (struct
Node*)malloc(sizeof(struct Node));
    newNode->data = data;

    // Insert after the given node
    newNode->next = temp->next; // new
-> next points to next of given node
    temp->next = newNode;      // given
node -> next points to new node
}
```

C

 Copy code

```
int searchSLL(struct Node* head, int
key) {
    int pos = 1;                //
position counter
    struct Node* temp = head;

    // Traverse till end
    while (temp != NULL) {
        if (temp->data == key)
            return pos;        // key
found, return position

        temp = temp->next;
        pos++;
    }

    return -1;                  // key not
found
}
```

C

 Copy code

```
void traverseSLL(struct Node* head) {  
    struct Node* temp = head;  
  
    // Visit each node  
    while (temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL");  
}
```

◆ Delete at Beginning

C

 Copy code

```
struct Node* deleteBegSLL(struct Node*  
head) {  
    if (head == NULL)  
        return NULL;  
  
    struct Node* temp = head;  
    head = head->next;  
    free(temp);  
    return head;  
}
```

◆ Delete at Specific Position

C

 Copy code

```
struct Node* deletePosSLL(struct Node*
head, int pos) {
    if (pos == 1)
        return deleteBegSLL(head);

    struct Node* temp = head;
    for (int i = 1; i < pos - 1 &&
temp->next != NULL; i++)
        temp = temp->next;

    if (temp->next == NULL)
        return head;

    struct Node* del = temp->next;
    temp->next = del->next;
    free(del);
    return head;
}
```

◆ Delete at End

C

 Copy code

```
struct Node* deleteEndSLL(struct Node*
head) {
    if (head == NULL || head->next ==
NULL)
        return NULL;

    struct Node* temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;

    free(temp->next);
    temp->next = NULL;
    return head;
}
```

◆ Insert at Beginning

C

 Copy code

```
struct DNode* insertBegDLL(struct DNode*  
head, int data) {  
    struct DNode* newNode = (struct  
DNode*)malloc(sizeof(struct DNode));  
    newNode->data = data;  
    newNode->prev = NULL;  
    newNode->next = head;  
  
    if (head != NULL)  
        head->prev = newNode;  
  
    return newNode;  
}
```

◆ Insert at Specific Position

C

 Copy code

```
struct DNode* insertPosDLL(struct DNode*
head, int data, int pos) {
    if (pos == 1)
        return insertBegDLL(head, data);

    struct DNode* temp = head;
    for (int i = 1; i < pos - 1 &&
temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL)
        return head;

    struct DNode* newNode = (struct
DNode*)malloc(sizeof(struct DNode));
    newNode->data = data;
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;
    return head;
}
```

◆ Insert at End

C

 Copy code

```
struct DNode* insertEndDLL(struct DNode*
head, int data) {
    if (head == NULL)
        return insertBegDLL(head, data);

    struct DNode* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    struct DNode* newNode = (struct
DNode*)malloc(sizeof(struct DNode));
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = temp;
    temp->next = newNode;

    return head;
}
```

◆ Delete at Beginning

C

 Copy code

```
struct DNode* deleteBegDLL(struct DNode*
head) {
    if (head == NULL)
        return NULL;

    struct DNode* temp = head;
    head = head->next;

    if (head != NULL)
        head->prev = NULL;

    free(temp);
    return head;
}
```

◆ Delete at End

C

 Copy code

```
struct DNode* deleteEndDLL(struct DNode*
head) {
    if (head == NULL || head->next ==
NULL)
        return NULL;

    struct DNode* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->prev->next = NULL;
    free(temp);
    return head;
}
```

◆ Insert at Beginning

C

📋 Copy code

```
struct CNode* insertBegCLL(struct CNode*
head, int data) {
    struct CNode* newNode = (struct
CNode*)malloc(sizeof(struct CNode));
    newNode->data = data;

    if (head == NULL) {
        newNode->next = newNode;
        return newNode;
    }

    struct CNode* temp = head;
    while (temp->next != head)
        temp = temp->next;

    newNode->next = head;
    temp->next = newNode;
    return newNode;
}
```

◆ Insert at Specific Position

C

📋 Copy code

```
struct CNode* insertPosCLL(struct CNode*
head, int data, int pos) {
    if (pos == 1)
        return insertBegCLL(head, data);

    struct CNode* temp = head;
    for (int i = 1; i < pos - 1 &&
temp->next != head; i++)
        temp = temp->next;

    struct CNode* newNode = (struct
CNode*)malloc(sizeof(struct CNode));
    newNode->data = data;
    newNode->next = temp->next;
    temp->next = newNode;
    return head;
}
```

◆ Insert at End

C

 Copy code

```
struct CNode* insertEndCLL(struct CNode*  
head, int data) {  
    return insertPosCLL(head, data,  
100); // large position = end  
}
```

◆ Delete at Beginning

C

 Copy code

```
struct CNode* deleteBegCLL(struct CNode*  
head) {  
    if (head == NULL || head->next ==  
head)  
        return NULL;  
  
    struct CNode* temp = head;  
    struct CNode* last = head;  
  
    while (last->next != head)  
        last = last->next;  
  
    head = head->next;  
    last->next = head;  
    free(temp);  
    return head;
```



◆ Delete at End

C

 Copy code

```
struct CNode* deleteEndCLL(struct CNode*
head) {
    if (head == NULL || head->next ==
head)
        return NULL;

    struct CNode* temp = head;
    while (temp->next->next != head)
        temp = temp->next;

    free(temp->next);
    temp->next = head;
    return head;
}
```

```

void insertAfterDLL(struct DNode* head,
int key, int data) {
    struct DNode* temp = head;

    // Traverse list to find the given
node (key)
    while (temp != NULL && temp->data !=
key)
        temp = temp->next;

    // If given node not found
    if (temp == NULL) {
        printf("Node not found\n");
        return;
    }

    // Create new node
    struct DNode* newNode = (struct
DNode*)malloc(sizeof(struct DNode));
    newNode->data = data;

    // Link new node
    newNode->next = temp->next;
    newNode->prev = temp;

    // If not inserting at last position
    if (temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;
}

```

C

 Copy code

```
int searchDLL(struct DNode* head, int
key) {
    int pos = 1;
    struct DNode* temp = head;

    // Traverse forward
    while (temp != NULL) {
        if (temp->data == key)
            return pos;

        temp = temp->next;
        pos++;
    }

    return -1;    // key not found
}
```